K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Generative AI Lab

ENSP356

LAB FILE

Submitted By:

Ishika

2301420052

B.Tech CSE (Data Science)

Semester-6

Submitted To:

Ms. Lucky Verma

INDEX

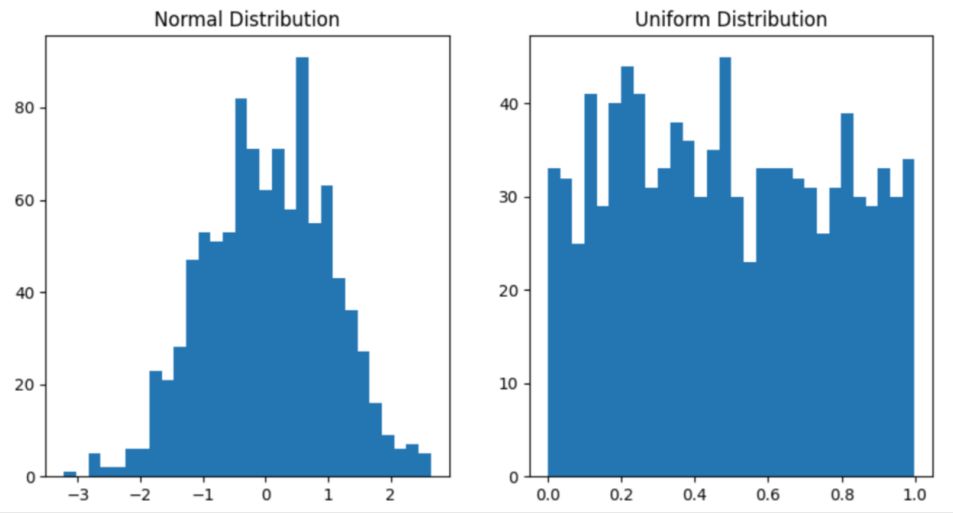
S.No	Title	Sign
1.	Experiment 1	
2.	Experiment 2	
3.	Experiment 3	
4.	Experiment 4	
5.	Experiment 5	
6.	Experiment 6	
7.	Experiment 7	
8.	Experiment 8	
9.	Experiment 9	
10.	Experiment 10	
11.	Project	

Experiment 1

Objective

To generate random samples from various probability distributions such as Normal Distribution and Uniform Distribution using Python.

```
import numpy as np
import matplotlib.pyplot as plt

# Normal Distribution
normal_data = np.random.normal(loc=0, scale=1, size=1000)

# Uniform Distribution
uniform_data = np.random.uniform(low=0, high=1, size=1000)

# Plotting
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.hist(normal_data, bins=30)
plt.title("Normal Distribution")

plt.subplot(1,2,2)
plt.hist(uniform_data, bins=30)
plt.title("Uniform Distribution")

plt.show()
```



Experiment 2

Objective

To implement Maximum Likelihood Estimation (MLE) for estimating parameters of a Normal Distribution.

```
import numpy as np
from scipy.stats import norm

# Generate synthetic data
data = np.random.normal(loc=5, scale=2, size=1000)

# MLE estimation
mu_estimate = np.mean(data)
sigma_estimate = np.std(data)

print("Estimated Mean:", mu_estimate)
print("Estimated Standard Deviation:", sigma_estimate)
```

```
Estimated Mean: 5.099456185140143
Estimated Standard Deviation: 2.0146799231268404
```

Experiment 3

Objective

To train a neural network on the MNIST dataset using backpropagation and stochastic gradient descent.

```python
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

# Load dataset
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# Build model
model = models.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='sgd',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train model
model.fit(x_train, y_train, epochs=5)

# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_acc)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py
  super().__init__(**kwargs)
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8391 - loss: 0.6364
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9060 - loss: 0.3377
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9188 - loss: 0.2879
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9279 - loss: 0.2577
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9342 - loss: 0.2349
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9372 - loss: 0.2201
Test Accuracy: 0.9372000098228455
```

Experiment 4

Objective

To train a Generative Adversarial Network (GAN) on the MNIST dataset and monitor generator and discriminator losses.

```python
import tensorflow as tf
from tensorflow.keras import layers

# Simple Generator
generator = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(100,)),
    layers.Dense(784, activation='sigmoid')
])

# Simple Discriminator
discriminator = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(1, activation='sigmoid')
])

print(generator.summary())
print(discriminator.summary())
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: U
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 128)	12,928
dense_3 (Dense)	(None, 784)	101,136

```
Total params: 114,064 (445.56 KB)
Trainable params: 114,064 (445.56 KB)
Non-trainable params: 0 (0.00 B)
None
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 128)	100,480
dense_5 (Dense)	(None, 1)	129

```
Total params: 100,609 (393.00 KB)
Trainable params: 100,609 (393.00 KB)
Non-trainable params: 0 (0.00 B)
None
```

Experiment 5

Objective

To implement a flow-based generative model using a deep learning framework.

```python
import tensorflow as tf
from tensorflow.keras import layers

# Simple flow-based model
model = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10)
])

model.summary()
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 64)	704
dense_7 (Dense)	(None, 64)	4,160
dense_8 (Dense)	(None, 10)	650

```
Total params: 5,514 (21.54 KB)
Trainable params: 5,514 (21.54 KB)
Non-trainable params: 0 (0.00 B)
```

Experiment 6

Objective

To implement evaluation metrics using appropriate Python libraries.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

y_true = [1, 0, 1, 1, 0]
y_pred = [1, 0, 1, 0, 0]

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
```

```
Accuracy: 0.8
Precision: 1.0
Recall: 0.6666666666666666
```

Experiment 7

Objective

To evaluate the trained model's performance on a validation dataset.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

# Load dataset
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

# Model
model = models.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(x_train, y_train, epochs=3)

# Evaluate
loss, accuracy = model.evaluate(x_test, y_test)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37
  super().__init__(**kwargs)
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - accuracy: 0.9127 - loss: 0.3110
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9568 - loss: 0.1511
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9679 - loss: 0.1095
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9663 - loss: 0.1119
Validation Loss: 0.11189308017492294
Validation Accuracy: 0.9663000106811523
```

Experiment 8

Objective

To build an Encoder and Decoder architecture for a Variational Autoencoder (VAE).

```
import tensorflow as tf
from tensorflow.keras import layers

# Encoder
encoder = tf.keras.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(64, activation='relu'),
    layers.Dense(2)
])

# Decoder
decoder = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(2,)),
    layers.Dense(784, activation='sigmoid'),
    layers.Reshape((28,28))
])

print(encoder.summary())
print(decoder.summary())
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: U
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_11 (Dense)	(None, 64)	50,240
dense_12 (Dense)	(None, 2)	130

```
Total params: 50,370 (196.76 KB)
Trainable params: 50,370 (196.76 KB)
Non-trainable params: 0 (0.00 B)
None
Model: "sequential_6"
```

Layer (type)	Output Shape	Param #
dense_13 (Dense)	(None, 64)	192
dense_14 (Dense)	(None, 784)	50,960
reshape (Reshape)	(None, 28, 28)	0

```
Total params: 51,152 (199.81 KB)
Trainable params: 51,152 (199.81 KB)
Non-trainable params: 0 (0.00 B)
None
```

Experiment 9

Objective

To implement an autoregressive model using a deep learning framework.

```
import tensorflow as tf
from tensorflow.keras import layers

model = tf.keras.Sequential([
    layers.Embedding(input_dim=1000, output_dim=64),
    layers.SimpleRNN(128),
    layers.Dense(1000, activation='softmax')
])

model.summary()
```

```
Model: "sequential_7"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense_15 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

Experiment 9

Objective

To analyze loss functions used in GAN training such as adversarial loss and feature matching loss.

```
import numpy as np

# Example adversarial losses
generator_loss = np.random.random(10)
discriminator_loss = np.random.random(10)

print("Generator Losses:", generator_loss)
print("Discriminator Losses:", discriminator_loss)
```

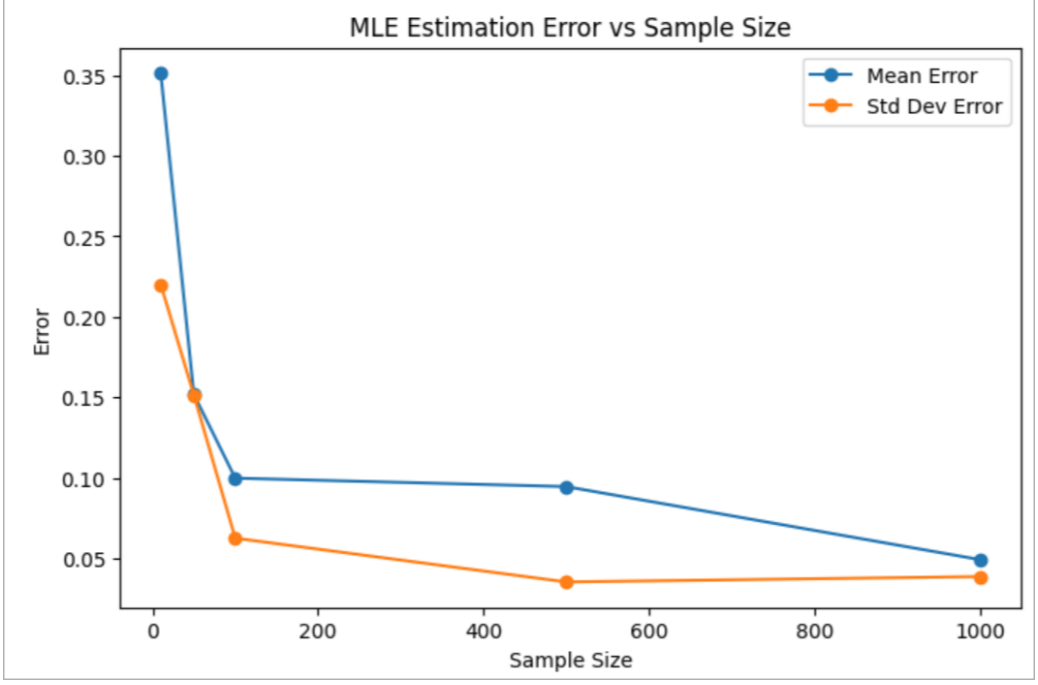
```
Generator Losses: [0.53862857 0.06581242 0.20627351 0.92229568 0.16519238 0.26195138
 0.6336228  0.82633348 0.52873075 0.00462875]
Discriminator Losses: [0.10988595 0.03969079 0.98437639 0.77960835 0.1258494  0.58068791
 0.95933266 0.290364   0.7476657  0.39646885]
```

Project: Parameter Estimation for a Probability Distribution

Objective

To implement the Maximum Likelihood Estimation (MLE) algorithm for estimating the parameters of a Normal Distribution using a synthetic dataset and compare the estimated parameters with the true parameters for different dataset sizes.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# True parameters of Normal Distribution
true_mean = 5
true_std = 2

# Different dataset sizes
sample_sizes = [10, 50, 100, 500, 1000]

results = []

for size in sample_sizes:

    # Generate synthetic dataset
    data = np.random.normal(loc=true_mean, scale=true_std, size=size)

    # MLE Estimates
    estimated_mean = np.mean(data)
    estimated_std = np.std(data)

    # Estimation Errors
    mean_error = abs(true_mean - estimated_mean)
    std_error = abs(true_std - estimated_std)

    # Store results
    results.append([
        size,
        estimated_mean,
        estimated_std,
        mean_error,
        std_error
    ])
```

```python
# Create DataFrame
df = pd.DataFrame(results, columns=[
    "Sample Size",
    "Estimated Mean",
    "Estimated Std Dev",
    "Mean Error",
    "Std Dev Error"
])

print(df)

# Plot Errors
plt.figure(figsize=(8,5))

plt.plot(sample_sizes, df["Mean Error"], marker='o', label="Mean Error")
plt.plot(sample_sizes, df["Std Dev Error"], marker='o', label="Std Dev Error")

plt.xlabel("Sample Size")
plt.ylabel("Error")
plt.title("MLE Estimation Error vs Sample Size")
plt.legend()

plt.show()
```

	Sample Size	Estimated Mean	Estimated Std Dev	Mean Error	Std Dev Error
0	10	4.648738	1.779892	0.351262	0.220108
1	50	4.848168	1.848451	0.151832	0.151549
2	100	4.900123	1.937378	0.099877	0.062622
3	500	4.905374	1.964572	0.094626	0.035428
4	1000	5.049270	2.038765	0.049270	0.038765